Simplifying the K215 Correctness Proofs

A shared implication basis, direct seed semantics,
and one Rocq enumerator-completeness argument

ProveIt proof-refactoring study

October 21, 2026

Abstract

The maintained K215 development already contains substantial proof compression. Further simplification should therefore target repeated mathematical arguments and unnecessary intermediate constructions. This report supplies three compatible component proposals against the exact main-branch revision `57541fe0cfe3af62b4cb58747d8d1d7c16b338d5`. First, an existing six-goal implication derivation is integrated with the maintained producer, replacing the separate unrestricted derivation: the two Lean proof files shrink from 811 lines and 42,454 bytes to 401 lines and 18,243 bytes. Second, a previously proposed direct named-formula seed construction is implemented and checked. It removes the preliminary indiscernibility quotient from the Lean endpoint's dependency path. Third, a conservative Rocq refactor shares candidate progress, run-prefix transport, and boundary completeness while retaining every old theorem interface. The archive includes patches, source snapshots, compiler receipts, independent source reviews, and reproduction scripts. These are checked component and generic-endpoint results; the complete machine-specific Lean and Rocq endpoints have not been rebuilt for this packet.

1 Recommendation and measured result

Adopt proof sharing at the logical interface first, then shorten the lower semantic route by constructing the full seed directly on the original model. The strongest newly checked mathematical change is the latter: the completeness theorem already supplies satisfaction of the original named formulas, including literal equality. There is no need to pass through an equality-free model and an indiscernibility quotient merely to recover those formulas' seed instances.

The shared implication idea and the direct-seed outline were already present in the repository's research material. This packet's contribution is current-source implementation, maintained-producer synchronization, explicit compatibility checks, and fresh kernel checking. It does not claim a new smallest Turing machine or a new logical axiom basis. The provenance distinctions matter because the most recent merges deliberately preserved the checked production files while retaining later proposals as inert inputs [2, 3, 4].

The semantic refactor keeps the old quotient modules available to legacy clients. Consequently, its six-file maintenance set grows by 6,795 bytes even though the K215 endpoint imports 7,534 fewer source bytes. This is a shorter endpoint argument and dependency boundary, not a claim that every repository-wide size measure decreases. The two implication-file saving and semantic closure saving are additive within the selected endpoint closure. The various historical alternative implication proposals are not additive.

Scope	Before	Proposed	Meaning
Two Lean implication files, lines	811	401	410 fewer
Two Lean implication files, bytes	42,454	18,243	57.0% fewer
Those files plus producer, bytes	52,651	28,760	23,891 fewer
Direct-seed endpoint closure, modules	179	178	One fewer net
Direct-seed closure, source bytes	2,129,439	2,121,905	7,534 fewer
Both Lean changes, closure bytes	2,129,439	2,097,694	31,745 fewer
Two Rocq enumerator files, lines	410	403	Old APIs kept
Two Rocq enumerator files, bytes	18,100	17,833	267 fewer

Table 1: Different scopes are reported separately. Dependency-closure reductions are static source measurements, not elapsed-time or proof-term-size measurements.

2 The exact baseline and preservation contract

2.1 Source files, not commit subjects, determine the baseline

The review is pinned to the full commit hash given in the abstract. Its maintained Lean sources are the phase-twelve checkpoint: 179 project modules, including 167 BusyBeaver modules, and 2,129,439 selected source bytes. Independently recomputing the source bindings finds all 199 of the checkpoint's primary and compatibility source hashes unchanged. This source census is not a new full build.

In particular, the checked `DominatedImplication.lean` has 287 lines and 13,723 bytes, while `Lukasiewicz.lean` has 524 lines and 28,731 bytes. The 300-line core and 101-line unrestricted adapter from branch revision `49e99c5b7cc6c2e164331961de28bad3904361fd` are preserved under the phase-twelve upstream-merge inputs. They are not the production files at the pin. The merge record itself explains that producer validation and Lean integration remain separate acceptance steps [3].

The supplied audit script reproduces the import closure, source hashes, and public-declaration comparison. The proposed shared pair affects 33 modules transitively in the original endpoint closure. The expensive site batches and source-correspondence certificate modules are outside that affected set.

2.2 What is preserved

The literal 215-state transition table, blank-tape convention, source program, enumerator transition function, numerical formula encoding, and numerical calculus are unchanged. The existing endpoint is of the form

$$\mathrm{BlankHalts}(M_{215}) \iff \mathrm{BProv}(\mathrm{ZFCax}_s, [], \bot).$$

Negating both sides yields the same nonhalting/consistency equivalence. The intermediate equivalence between numerical derivability of `Code254.F` and ZF refutability, and the existing ZF–ZFC refutation bridge, retain their types.

As the maintained checkpoint states, this is an external proof-assistant theorem. The packet does not provide a new arithmetized derivation inside ZFC or PA. It also does not replace the machine-specific validation evidence with a claim of a fresh full endpoint build. The actual acceptance scope is given in Section 6.

3 One implication derivation for both logical layers

3.1 The abstract contract

Let A be an arbitrary carrier, let $x \to y$ be an arbitrary binary operation on A, and let $\preceq$ be a relation satisfying only

$$u \preceq u, \qquad u \preceq x \Longrightarrow u \preceq (x \to y), \qquad u \preceq y \Longrightarrow u \preceq (x \to y).$$

No injectivity, transitivity, semantics, truth element, or numerical representation is required. Define

$$\rho_8(a, b, c, u) = ((c \to u) \to a) \to ((a \to c) \to (b \to c)).$$

For a predicate $T : A \to \mathrm{Prop}$, the restricted interface supplies modus ponens and $T(\rho_8(a, b, c, u))$ whenever $u \preceq a$.

The preserved six-goal derivation establishes unrestricted ρ_8, composition, antecedent lifting, and the classical implicational basis:

$$\begin{aligned} K(a, b) &= a \to (b \to a), \\ P(a, b) &= ((a \to b) \to a) \to a, \\ S(a, b, c) &= (a \to (b \to c)) \to ((a \to b) \to (a \to c)). \end{aligned}$$

Each internal inference is an explicit modus-ponens application to already proved instances. Each restricted-row guard is justified by reflexivity and insertion into an implication operand, or by a local guard hypothesis. The proof is therefore representation independent in exactly the sense required by the existing interface.

Proposition 3.1 (Unrestricted adapter)**.** *An unrestricted ρ_8 calculus with modus ponens is an instance of the restricted interface with $u \preceq a$ defined to be* True*.*

Proof. Reflexivity and both insertion rules hold trivially. The restricted-row field ignores its guard and invokes the supplied unrestricted row. Applying the generic theorems gives K, Peirce, composition, and S with the old unrestricted types. □

This small interpretation is the entire new unrestricted proof. It replaces `cd_0` through `cd_106`; there is no second detachment development to maintain. The numerical adapter remains byte-for-byte unchanged and continues to instantiate the same generic interface with the repository's coding operation and natural-number domination relation. Its legacy truth field remains available; the generic theorem does not use it.

3.2 Proof accounting and interface preservation

The raw shared derivation has 58 private modus-ponens nodes. Two final aliases are inlined into their public results, leaving 56 private nodes and three public modus-ponens applications, including the lifting wrapper common to the comparison. Thus both raw and rendered sources contain 59 explicit MP applications when that wrapper is counted. The six goals do not add a second proof tree.

The public `Language`, `Restricted`, `rho8`, `rho8_all`, `composition`, and `lift_right` interfaces remain unchanged. The generic module gains K, Peirce, and S exports. The unrestricted

`Implicational` structure, its field order, and all four basis theorem types are preserved. The final Tarski namespace section is byte-identical. Fourteen original public declaration headers compare identically as normalized source text; separate typed Lean clients check elaboration, not just text.

The core grows by 13 lines while the unrestricted module shrinks by 423 lines. The relevant unit of comparison is both files together. The resulting 410-line reduction is neither a minimality theorem for Hilbert proofs nor a bound on fully expanded proof terms.

3.3 Synchronizing the maintained producer

Changing only the Lean files would leave a fragile integration: the maintained producer would still pin and regenerate the older three-goal core. The delivered patch therefore changes `generate_dominated_implication_lean.py` as well. Its successor reads the raw six-goal source and derivation ancestry already preserved under phase twelve. It does not execute the historical search program or historical renderer.

The producer checks the pinned input hashes, replays MP and guard obligations in the raw source, performs the two explicit alias substitutions, and replays the inlined source with all six public targets. It checks 23 restricted-row uses in each source form. The separate condensed-detachment text is pinned ancestry data; it is not represented as having been parsed by that source reader. The new source reader is a diagnostic and reproducibility check. Lean's kernel is the authority for the exported theorems.

The output is exactly the preserved 14,146-byte core and 4,097-byte unrestricted adapter, plus the unchanged 2,680-byte numerical adapter. The producer grows by 320 bytes. Counting the two changed proof files and the producer gives a net reduction of 23,891 bytes and 401 lines.

The authoritative maintenance command after applying the patch is

```
python3 Computability/BusyBeaver/Research/ZFC/k225-formal/\
  generate_dominated_implication_lean.py --check
```

Use `-out DIR` to regenerate into a separate Lean source root. The preserved core's historical header names the research producer; the command above is the maintained successor and intentionally uses the preserved six-goal inputs.

4 Direct full seed semantics on the original domain

4.1 Why the old quotient can be bypassed

The old lower chain first derives seed instances for equality-free formulas, then forms an indiscernibility quotient, then transfers the seed property to all named formulas on that quotient. That route is valid, but its starting interface is stronger than the route uses.

Write $\mathrm{th}(f)$ for the numerical calculus's equality interpretation of a named formula, and set

$$\mathrm{DerivableNF}(f) \;:\Longleftrightarrow\; \mathcal{C}_{222}(\mathrm{enc}(\mathrm{th}(f))).$$

The syntactic premise of `KEind222.c222_complete` mentions the interpreted code. Its semantic premise, however, supplies

$$H_T: \quad \forall f \in \mathrm{DerivableNF}\ \forall e, \mathrm{Sat}(E, e, f).$$

This is satisfaction of the *original* named formula f, with literal equality. It is not merely satisfaction of $\mathrm{th}(f)$. The direct construction uses that exact interface. It never assumes that satisfaction of an equality translation can be reflected back on the original domain.

4.2 The seed property and named bundle

For a membership relation E and a binary relation R, write

$$x \subseteq_E y \iff \forall w\,(E(w,x) \Rightarrow E(w,y))$$

and define the request condition

$$\mathrm{Req}_R(x,C) \iff \forall y\,[R(x,y) \Rightarrow \exists z\,(R(x,z) \wedge E(z,C))].$$

The existing Glazer seed condition can then be written compactly as

$$\begin{aligned}\mathrm{Seed}_E(R,A) \iff \exists C\,[\,&\mathrm{Req}_R(A,C) \wedge \\ &\forall y\,(E(y,C) \Rightarrow \exists x\,[R(x,y) \wedge \forall x'\,(x' \subseteq_E y \Rightarrow \mathrm{Req}_R(x',C))])\,].\end{aligned}$$

This is the repository's `SeedProp`; no new set-theoretic principle is introduced by this notation.

The named formula `bundleNF a b k m` expresses that condition using input name a, output name b, private container name k, and relation matrix m. The key new semantic lemma is the named analogue of the existing numerical bundle lemma.

Lemma 4.1 (Named bundle semantics)**.** *Suppose $a \neq b$, $k \neq a$, $k \neq b$, and k is not free in m. Then*

$$\mathrm{Sat}(E,e,\mathrm{bundleNF}(a,b,k,m)) \iff \mathrm{Seed}_E\big((x,y) \mapsto \mathrm{Sat}(E,e[a:=x][b:=y],m),e(a)\big).$$

No irreflexivity hypothesis on E is required for this lemma.

Proof. Expand the named request macro to the request condition and the closure macro to the second conjunct of the seed condition. The auxiliary variable-relation implication is true because k differs from a and b. Classical normalization of a negated universal implication supplies a container C. Commute updates at distinct names and use freshness to remove the update $e[k := C]$ from the matrix's satisfaction predicate. The two resulting conjuncts are the seed conjuncts in reversed order; commutativity of conjunction finishes the proof.

Named falsity is actual falsity, which is why this lemma needs no irreflexivity premise. The existing numerical bundle theorem still needs its original irreflexivity hypothesis because its decoded falsity code has a different representation. That numerical theorem is left unchanged. □

4.3 A fresh private marker without renaming

For an arbitrary named formula ψ, including one with equality, put

$$m = \neg\neg\psi, \qquad k = \mathrm{enc}(\mathrm{th}(m)), \qquad B_\psi = \mathrm{bundleNF}(0,1,k,m).$$

The following facts justify the construction.

Lemma 4.2 (Freshness and role bounds)**.** *The marker satisfies* $1 < k$ *and* $\neg\,\mathrm{Free}(k, m)$.

Proof. First show by structural induction that

$$\mathrm{Free}(i, f) \Longrightarrow \mathrm{Free}(i, \mathrm{th}(f)).$$

The equality case uses the freshness of the auxiliary name in the membership indiscernibility formula. Its two original free names remain free. The universal case preserves the same binder. This is a free-support statement, not a claim that every syntactic occurrence survives translation.

The existing code-bound theorem says that a name occurring in a formula is strictly smaller than its encoding. If k were free in m, free-support preservation and that code bound would imply $k < \mathrm{enc}(\mathrm{th}(m)) = k$, a contradiction.

For the role bound, translation commutes with negation. The outer negation in m is an implication to the falsity code, and the existing right-projection bound for the pairing operation makes its encoding greater than one. The delivered proof obtains this directly from `le_P_right`; it does not add another positivity induction over ψ. Double negation simultaneously preserves the relation defined by ψ. □

Lemma 4.3 (The required bundle is derivable)**.** B_ψ *belongs to* DerivableNF.

Proof. Every atom introduced by the named bundle constructors is a membership atom. Consequently equality translation commutes with those constructors:

$$\mathrm{th}(\mathrm{bundleNF}(a, b, k, m)) = \mathrm{bundleNF}(a, b, k, \mathrm{th}(m)).$$

With $k = \mathrm{enc}(\mathrm{th}(m))$, the existing encoding identity gives

$$\mathrm{enc}(\mathrm{th}(B_\psi)) = \mathrm{bundle222}(0, 1, k).$$

Since $0 < k$ and $1 < k$, this is an enabled `C222.bundle` instance. By the definition of DerivableNF, the claim follows. □

Theorem 4.4 (Direct full seed)**.** *Let* E *be irreflexive and suppose* H_T *gives satisfaction of every formula in* DerivableNF *under every valuation. Then* E *has the seed property for every named relation on its original domain, including relations defined using literal equality.*

Proof. For ψ and a valuation e, apply H_T to B_ψ. The preceding lemmas allow application of named bundle semantics, producing the seed property for the relation defined by $m = \neg\neg\psi$. Classical double-negation elimination gives pointwise equivalence with the relation defined by ψ. The existing `seedProp_congr` transports the seed property across that equivalence. Combine these instances with the supplied irreflexivity hypothesis to obtain `FullSeed mem`. □

4.4 The actual endpoint change

The lower endpoint now uses the original domain and its existing valuation:

```
have hfull := fullSeed_direct_of_derivable mem hirr hT
exact zf_consistent_of_formSeed (formSeed_of_fullSeed hfull)
  hfull.irr (e 0) hf
```

The witness `e 0` replaces the old quotient representative `KIndisc222.mk mem (e 0)`. Thus no inhabited-domain premise is dropped. The full lower theorem's statement is unchanged.

To make the dependency reduction real, `KSeedInterface222` holds the named bundle syntax, its encoding identity, `DerivableNF`, and `FullSeed`, retaining their original namespaces and types. `KSeedDirect222` supplies the new proof. The old equality-free and indiscernibility modules import the shared interface and still compile as compatibility clients.

Three modules leave the selected endpoint closure: `Code254Models`, `KSeedModel222`, and `KIndisc222`. Two enter: `KSeedInterface222` and `KSeedDirect222`. The resulting reduction is 150 selected source lines and 7,534 bytes. The first quotient is bypassed; the later tagged-domain bisimulation quotient, well-founded part, and ZF model construction remain unchanged. The same architectural idea already appears in the Rocq lower proof, so no corresponding second deletion is claimed there [4, 5].

5 One Rocq enumerator-completeness argument

5.1 The common command boundary

The maintained Rocq lane separately proves that enumeration reaches later candidates from its zero start and from its bootstrap seed. It also repeats the run-prefix transport and final completeness argument. The refactor makes the shared hypothesis explicit: a state is at a command boundary when its pending command field `pl` is zero [6].

For an arbitrary boundary state s and $j \in \mathbb{N}$, the shared reachability theorem gives

$$\text{halts_from}(s) \quad \vee \quad \exists n, t\, [\text{run}(n, s) = \text{Some}(t) \wedge \text{pl}(t) = 0 \wedge \text{np}(t) = \text{np}(s) + j].$$

The proof is the existing generalized seed induction moved to the common compiler module. It is complemented by one shared lemma transporting halting backward along an already proved finite run.

Theorem 5.1 (Boundary completeness)**.** *If* $\text{pl}(s) = 0$ *and* `thm222 1`*, then* `halts_from s`*.*

Proof. Let $Y = \text{np}(s)$ and compile a refutation into a command block L. Choose the reset-prefixed candidate code

$$N = P(3, P(Y, P(1, \text{enc_tail}(L +\!\!+ \text{SUFFIX})))).$$

The inherited pairing inequalities give $Y \leq N$. Apply boundary reachability with $j = N - Y$. If the search has already halted, the conclusion follows. Otherwise it reaches a boundary state with candidate N. The existing `program_halts` contract runs the reset-prefixed refutation from every remaining working-register tuple. Compose that halting run with the prefix. □

This is a completeness theorem. It does not claim that arbitrary initial working registers satisfy the invariant needed for soundness. That separation is why the generalization can have the small hypothesis `pl seed = 0` without weakening the established consistency endpoint.

The old zero-start and bootstrap completeness theorems become direct specializations. All 46 original theorem headers are preserved, including the old drain, progress, and prefix helpers. The maintained templates, generated namespace variants, and corresponding manifest hashes are synchronized.

5.2 Scope of the improvement

The two touched proof files shrink by only seven lines and 267 bytes. Retaining every old helper interface costs compatibility wrappers, so this proposal is primarily a reduction in repeated arguments. The repository already contains a more invasive arbitrary-state rank proposal; that proposal removes some of those interfaces. The present component is a conservative alternative, not a new claim of scheduler-theorem novelty [7].

6 Verification and the remaining acceptance boundary

6.1 Completed checks

The final joint Lean validation freshly compiles 116 project modules under Lean 4.32.0 in one clean replay into a new output directory, using no precompiled project objects. It checks the shared implication pair, both new seed modules, the modified generic `KEndpoint`, retained legacy clients, and typed compatibility audits. The receipt records 26 selected named assumption reports. The distributed standalone reproducer was used for this complete replay.

The new generic implication results report no axioms. The named semantic and generic endpoint reports use subsets of the existing standard Lean axioms `propext`, `Classical.choice`, and `Quot.sound`. No new axiom, `sorry`, or unchecked native decision is introduced. The fresh direct-proof and audit modules have zero warnings; 18 preexisting linter warnings occur in unchanged upstream sources. Exact source, object, and log hashes are recorded with the named reports.

For Rocq, the baseline and candidate each freshly compile an eleven-module component closure with Coq 8.18.0 and OCaml 4.14.1. The baseline's nine typed interface reports and the candidate's twelve reports are closed under the global context. Both recursive `coqchk` runs succeed. The build copies change only the `From Stdlib Require` spelling to `From Coq Require` for this toolchain; distributed production files retain their original spelling.

Source checks additionally confirm that the patches apply at the pinned revision, the producer reproduces the intended Lean sources exactly, the numerical adapter is unchanged, all Rocq original theorem headers remain identical, and the generated Rocq files match the maintained templates and provenance headers.

6.2 What these checks do not establish

The joint Lean replay includes the generic machine-to-ZFC endpoint, not a fresh build of `KEndpoint215` and its complete machine-specific closure. The Rocq checks cover the compiler/seed component, not the complete production Rocq 9.1 machine-to-ZFC build. Existing machine and decoder receipts are not relabeled as new checks. Import-count and source-size measurements are not runtime benchmarks.

The final adoption gate is to rebuild the affected descendants and the actual machine-specific endpoints with the production toolchains, then compare their exact theorem types and assumption profiles. The unchanged site and source certificates may be reused only through the repository's established exact source/dependency/object binding procedure. This packet leaves the older accepted receipts intact.

7 Integration plan

Use an isolated branch based on the pinned revision and apply the three patches in the supplied order. The combined patch contains the same changes and may be used instead of the three individual patches.

1. Apply the shared-basis patch. Run the maintained producer's `-check` mode and the focused typed API check. This changes two Lean proof files and their maintained producer; it does not change the numerical adapter.
2. Apply the direct-seed patch. Rebuild the new interface and direct proof, the lower-seed client, the generic endpoint, and both legacy modules. The joint standalone replay in this archive already verifies this combination with the shared implication pair.
3. Apply the Rocq enumerator patch. Regenerate the two namespace variants through the normal production pipeline and check the manifest bindings. The archive's component replay checks both baseline and proposed interfaces.
4. Run the production machine-specific endpoint checks and preserve their new source/dependency/object receipts. Only after this step should the changes be labeled adopted at the machine endpoint.

The archive includes source snapshots for offline component reproduction. It does not contain compiled Lean or Rocq objects. The preserved phase-twelve inputs used by the producer remain at their established repository paths; no historical derivation is overwritten. Read the top-level `README.md` for exact commands and `MANIFEST.sha256` for delivered file hashes.

8 Further simplification questions

8.1 Share named and numerical bundle normalization

The direct proof intentionally follows the already understood named-bundle normalization before attempting a larger abstraction. The named and decoded versions still share update-commutation and request/closure reasoning. Can one small semantic interface capture both while keeping explicit the difference between primitive named falsity and decoded numerical falsity? A successful abstraction must preserve the numerical irreflexivity premise and the coincident role cases needed by the upper chain. Merely deleting those premises would be incorrect.

A second possible contraction is to construct `FormSeed` directly at named roles 3 for input and 1 for output, matching `Code254GIC.names`. This could remove the extra input-name wrapper used by `formSeed_of_fullSeed`. It needs a separate compatibility split and a measured net-source comparison; it is not implemented in this packet.

8.2 Replace whole-site execution by reusable primitive suffixes

The repository already has decoder-prefix and semantic-coverage proposals. The remaining substantial task is to prove and instantiate the primitive and return suffix contracts for every guarded outcome. The maintained site proof covers 4,751 leaves and 6,176 outcomes. Factoring its repeated execution should retain both decrement polarities, exact consumed paths including zero padding,

overflow behavior, the gap and spare-register conditions, and positive physical progress for backward halting transfer. A compiled single-leaf example is not complete coverage. This packet makes no claim to have discharged that global suffix premise.

8.3 A single abstract scheduler interface across proof assistants

The new Rocq boundary theorem and the existing Lean generalized enumerator argument suggest a common interface: a boundary predicate, candidate-rank progress, an early-halting alternative, and an executable refutation for a candidate above the current rank. A further refactor could derive both zero-start and bootstrap results from this interface in both assistants. Measure source growth from the abstraction against eliminated duplicate proofs; the present Rocq result shows why preserving all legacy wrappers can make raw line savings small even when the mathematical structure improves.

8.4 Proof-DAG optimization with several exports

The shared six-goal derivation shows that optimizing one export in isolation misses useful sharing. Further search should minimize the union of dependencies of all required exports, and should count generated proof files, adapters, and the maintained producer together. Proof-search output must continue to be replayed by the assistant kernels. No optimality conclusion follows from the current 58-node derivation.

8.5 A separate internal arithmetic endpoint

If the long-term target requires an explicit internal formal derivation of the machine/consistency equivalence in a weak arithmetic theory or in ZFC, that is a separate development. The present refactors improve the external verified argument and make its interfaces clearer. They do not close the internalization obligation by themselves.

Attribution and reproducibility notes

The machine, numerical calculus, semantic model construction, condensed-detachment derivations, and original compiler proofs are inherited from ProveIt and the sources credited there. In particular, the shared implication basis is preserved branch work; the direct-seed construction implements the prior `sharing-interfaces` outline and follows an architecture already used in Rocq. The packet retains source attribution headers and the repository's MIT No Attribution license. New compilation receipts establish the stated local checks; historical receipts are identified as historical evidence rather than silently promoted.

References

[1] V. Reshetnikov and ProveIt contributors. *ProveIt*, pinned main revision `57541fe0cfe3`. https://github.com/VladimirReshetnikov/ProveIt/commit/57541fe0cfe3af62b4cb58747d8d1d7c16b338d5.

[2] ProveIt contributors. *Twelfth K215 proof-simplification checkpoint*. Pinned repository path `Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/phase12/README.md`. Source hashes independently checked by the supplied `audit_current.py`.

[3] ProveIt contributors. *Shared implication core and alternate semantic cut*. Pinned repository path `Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/phase12/upstream-merge/README.md`, with `merge.json` and preserved inputs. Proposed proof lineage: https://github.com/VladimirReshetnikov/ProveIt/commit/49e99c5b7cc6c2e164331961de28bad3904361fd.

[4] ProveIt research packet. *Direct full seed: remove the first quotient, not the second*. Pinned repository path `Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/sharing-interfaces/calculus/DIRECT_FULL_SEED.md`. Previously an audited, uncompiled Lean proposal; implemented and checked here.

[5] ProveIt contributors. *Complete222*, Rocq direct lower construction. Pinned repository path `Computability/BusyBeaver/Research/ZFC/k225-formal/rocq224/src/Complete222.v`. See `tau_fr`, `th_fr`, `seed_code_fresh`, and `Q_seed`.

[6] ProveIt contributors. *L1_CompPerm* and *SeedPerm*. Maintained templates under `Computability/BusyBeaver/Research/ZFC/k225-formal/rocqperm/`; generated K215 sources under `Computability/BusyBeaver/Research/ZFC/k225-formal/rocq/rocq-extension/coq/215/`.

[7] ProveIt research packet. *Checked transport and seed rank*. Pinned proposal directory `Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/checked-transport/rocq_seed_rank/`. Alternative arbitrary-state completeness refactor, credited separately.